Lab 8: Edge Detection - Python Code

Student: Abdullah Hejji ALnuwaysir
Student ID: 2240005501

Sobel | Prewitt | Canny

```python
from pathlib import Path
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Student information
STUDENT_NAME = "Abdullah Hejji ALnuwaysir"
STUDENT_ID = "2240005501"

# Read the supplied kitten image from the notebook's working directory.
image_path = Path("kitten.jpg")
if not image_path.is_file():
    raise FileNotFoundError(
        "kitten.jpg was not found. Keep the supplied image beside this notebook."
    )

img = cv2.imread(str(image_path), cv2.IMREAD_COLOR)
if img is None:
    raise ValueError("OpenCV could not read kitten.jpg.")

# OpenCV loads BGR; convert to RGB for plotting and grayscale for edge detection.
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

output_dir = Path("results")
output_dir.mkdir(exist_ok=True)


def to_uint8(response):
    """Normalize a gradient magnitude to the 0-255 display range."""
    return cv2.normalize(
        response, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)

print(f"Student: {STUDENT_NAME}")
print(f"Student ID: {STUDENT_ID}")
print(f"Input image: {image_path.name} ({img.shape[1]} x {img.shape[0]} pixels)")

# Sobel: first derivatives in the x and y directions.
sobel_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobel_x, sobel_y)
sobel = to_uint8(sobel_magnitude)

# Prewitt: manually defined 3 x 3 derivative kernels.
kernel_x = np.array(
    [[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64
)
kernel_y = np.array(
    [[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64
```

```
)
prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)
prewitt_magnitude = cv2.magnitude(prewitt_x, prewitt_y)
prewitt = to_uint8(prewitt_magnitude)

# Canny edge detection with the reference thresholds.
canny_100_200 = cv2.Canny(gray, 100, 200)

# Save the edge maps for later use or submission.
cv2.imwrite(str(output_dir / "sobel.png"), sobel)
cv2.imwrite(str(output_dir / "prewitt.png"), prewitt)
cv2.imwrite(str(output_dir / "canny_100_200.png"), canny_100_200)

print("Task 1 edge maps generated: Sobel, Prewitt, and Canny (100, 200).")

fig, axes = plt.subplots(2, 2, figsize=(12, 7.5))
panels = [rgb, sobel, prewitt, canny_100_200]
titles = [
    "Original kitten image",
    "Sobel gradient magnitude",
    "Prewitt gradient magnitude",
    "Canny edges (low=100, high=200)",
]

for ax, panel, title in zip(axes.flat, panels, titles):
    if panel.ndim == 3:
        ax.imshow(panel)
    else:
        ax.imshow(panel, cmap="gray", vmin=0, vmax=255)
    ax.set_title(title)
    ax.axis("off")

fig.suptitle("Lab 8 - Edge Detection | Abdullah Hejji ALnuwaysir | 2240005501", fontsize=12)
fig.tight_layout()
fig.savefig(output_dir / "task1_methods.png", dpi=180, bbox_inches="tight")
plt.show()
plt.close(fig)

threshold_pairs = [(30, 90), (50, 150), (100, 200)]
canny_maps = {}
edge_counts = []

fig, axes = plt.subplots(1, 3, figsize=(15, 4.7))
for ax, (low, high) in zip(axes, threshold_pairs):
    # Only thresholds change between runs.
    edges = cv2.Canny(gray, low, high)
    canny_maps[(low, high)] = edges

    count = int(np.count_nonzero(edges))
    percent = 100.0 * count / edges.size
```

```
    edge_counts.append((low, high, count, percent))

    ax.imshow(edges, cmap="gray", vmin=0, vmax=255)
    ax.set_title(
        f"Low={low}, High={high}\n{count:,} edge pixels ({percent:.2f}%)"
    )
    ax.axis("off")
    cv2.imwrite(str(output_dir / f"canny_{low}_{high}.png"), edges)

fig.suptitle("Canny Threshold Comparison - Kitten Image", fontsize=12)
fig.tight_layout()
fig.savefig(output_dir / "task2_canny_thresholds.png", dpi=180, bbox_inches="tight")
plt.show()
plt.close(fig)

print("Low | High | Edge pixels | Percentage of image")
print("----|------|-------------|--------------------")
for low, high, count, percent in edge_counts:
    print(f"{low:3d} | {high:4d} | {count:11,d} | {percent:7.2f}%")

# Display a concise results table in the notebook.
print("Threshold experiment summary")
for low, high, count, percent in edge_counts:
    print(
        f"Thresholds ({low}, {high}): {count:,} edge pixels, "
        f"covering {percent:.2f}% of the image."
    )
```